# fn sample_poisson

OpenDP

March 17, 2026

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of `fn sample_poisson` in `mod.rs`. This proof reduces the general Poisson sampler to repeated calls to `sample_poisson_0_1` on equal sub-means in $[0, 1)$, then sums the results.

## 1 Hoare Triple

### Precondition

#### Compiler-verified

- Argument `lambda` is of type `RBig`, a bignum rational

#### User-verified

`lambda` $\geq 0$

### Implementation

```
def sample_poisson(lmbda):
    if lmbda == 0:
        return 0

    pieces = floor(lmbda) + 1  #
    piece = lmbda / pieces      #

    total = 0
    remaining = pieces
    while remaining > 0:
        total += sample_poisson_0_1(piece)  #
        remaining -= 1

    return total  #
```

### Postcondition

**Theorem 1.1.** For any setting of the input parameter `lambda` such that the given preconditions hold, `sample_poisson` either returns `Err(e)` due to a lack of system entropy, or `Ok(out)`, where `out` is distributed as $Poisson$(`lambda`).

## 2 Proof

**Lemma 2.1.** `sample_poisson` only returns `Err(e)` when there is a lack of system entropy.

*Proof.* If `lambda` $= 0$, the function returns `Ok(0)` immediately.

Otherwise, the only fallible call is line 11, which invokes `sample_poisson_0_1`. By line 5, `pieces` $= \lfloor$`lambda`$\rfloor + 1$, so `pieces` is a positive integer. Therefore line 6 defines

$$\texttt{piece} = \frac{\texttt{lambda}}{\texttt{pieces}},$$

which satisfies $0 \leq$ `piece` $< 1$ because `pieces` $>$ `lambda` whenever `lambda` $> 0$. Hence the preconditions of `sample_poisson_0_1` hold.

By the definition of `sample_poisson_0_1`, any error it returns is due to a lack of system entropy. Therefore `sample_poisson` only returns `Err(e)` when there is a lack of system entropy. □

We now condition on not returning an error.

**Lemma 2.2.** Each invocation of `sample_poisson_0_1(piece)` on line 11 returns an independent random variable distributed as $Poisson(\texttt{piece})$.

*Proof.* By the proof of Lemma 2.1, the preconditions of `sample_poisson_0_1` hold for `piece`. Therefore each call returns a $Poisson(\texttt{piece})$ random variable by its postcondition. Independence follows from the fact that the calls are made sequentially with fresh randomness. □

**Lemma 2.3.** If $X_1, \ldots, X_m$ are independent and each $X_i \sim Poisson(\mu)$, then

$$X_1 + \cdots + X_m \sim Poisson(m\mu).$$

*Proof.* The probability generating function of a $Poisson(\mu)$ random variable is

$$G_X(z) = \mathbb{E}[z^X] = \sum_{k=0}^{\infty} e^{-\mu} \frac{\mu^k}{k!} z^k = e^{\mu(z-1)}.$$

Therefore, if $X_1, \ldots, X_m$ are independent and each distributed as $Poisson(\mu)$, the probability generating function of their sum is

$$\mathbb{E}[z^{X_1+\cdots+X_m}] = \prod_{i=1}^{m} \mathbb{E}[z^{X_i}] = \left(e^{\mu(z-1)}\right)^m = e^{m\mu(z-1)},$$

which is the probability generating function of $Poisson(m\mu)$. □

**Lemma 2.4.** If the outcome of `sample_poisson` is `Ok(out)`, then `out` is distributed as $Poisson(\texttt{lambda})$.

*Proof.* If `lambda` $= 0$, the function returns 0, which is $Poisson(0)$.

Assume now that `lambda` $> 0$. Let

$$m := \lfloor \texttt{lambda} \rfloor + 1 \quad \text{and} \quad \mu := \frac{\texttt{lambda}}{m}.$$

By Lemma 2.2, each iteration of the loop on line 11 adds an independent $Poisson(\mu)$ random variable to `total`. Since the loop runs exactly $m$ times, `out` is the sum of $m$ independent $Poisson(\mu)$ random variables.

By Lemma 2.3,

$$\texttt{out} \sim Poisson(m\mu) = Poisson(\texttt{lambda}).$$

Thus `out` has the claimed distribution. □

*Proof of Theorem 1.1.* Holds by Lemma 2.1 and Lemma 2.4. □